

06_BACKEND.md

TASK 06

BACKEND DEVELOPMENT

Status

Mandatory

Owner

Backend Lead

Supporting Guides

Project Director

Security Architect

Solution Architect

Database Architect

QA & Security Lead

Principal Engineer

Estimated Time

5 to 8 Days

Dependencies

00_PROJECT_CONTEXT.md

00_ENGINEERING_RULES.md

00_PROJECT_MEMORY.md

07_BACKEND_LEAD.md

03_ARCHITECTURE.md

04_DATABASE.md

05_UI_DESIGN.md

Mission

Develop the complete backend for the Secure Dance Academy Management System.

Implement every business rule.

Protect every endpoint.

Ensure every request is validated, authenticated and authorized.

Produce production-quality backend code suitable for deployment.

Objectives

Implement all backend modules.

Create secure APIs.

Implement business logic.

Protect sensitive information.

Maintain clean architecture.

Write maintainable code.

Achieve high test coverage.

Development Principles

Follow:

Clean Architecture

Feature-Based Architecture

SOLID

DRY

KISS

YAGNI

Secure by Design

Least Privilege

Every module should remain independent.

Technology Stack

Framework

Next.js 15

Language

TypeScript

API

Route Handlers

Server Actions

ORM

Prisma

Database

Supabase PostgreSQL

Authentication

Supabase Auth

Validation

Zod

Password Hashing

bcrypt

Environment

dotenv

Backend Modules

Implement every module completely.

Authentication

Users

Roles

Permissions

Artists

Parents

Coaches

Attendance

Performance

Competitions

Medical Records

Injuries

Activities

Reports

Dashboard

Notifications

Audit Logs

Settings

Each module must contain

Routes

Services

Repositories

Validation

Types

Tests

Documentation

Authentication Module

Implement

Register

Login

Logout

Forgot Password

Reset Password

Session Refresh

Email Verification

Profile

Password Change

Session Management

Remember Me

Secure Cookies

Protect every authentication flow.

Authorization Module

Implement RBAC.

Administrator

Coach

Parent

Artist

Every endpoint must verify

Authentication

Role

Permission

Ownership

Context

Never trust frontend permissions.

User Management

Implement

Create User

Update User

Delete User

View User

Assign Role

Deactivate User

Restore User

Search Users

User Filters

Audit Actions

Artist Module

Implement

Create Artist

Update Artist

Delete Artist

View Artist

Medical Information

Emergency Contacts

Parent Assignment

Coach Assignment

Performance History

Profile Photo

Status Management

Parent Module

Implement

Create Parent

Update Parent

Assign Children

Emergency Contact

Profile

Notifications

Attendance View

Performance View

Parents must only access their own children.

Coach Module

Implement

Create Coach

Assign Artists

Attendance

Performance Updates

Medical Notes

Reports

Dashboard

Coach Profile

Attendance Module

Implement

Record Attendance

Edit Attendance

View Attendance

Attendance Reports

Attendance Statistics

Attendance Dashboard

Duplicate Prevention

Audit Logging

Performance Module

Implement

Training Scores

Competition Results

Rankings

Achievements

Statistics

Progress Tracking

Charts

Historical Records

Injury Module

Implement

Record Injury

Recovery Status

Medical Notes

Clearance

History

Restricted Access

Audit Logging

Medical information requires additional protection.

Activities Module

Implement

Create Activities

Schedules

Participation

Attendance

Calendar

Notifications

History

Dashboard Module

Implement

Administrator Dashboard

Coach Dashboard

Parent Dashboard

Artist Dashboard

Statistics

Recent Activity

Quick Actions

Alerts

Reports Module

Generate

Attendance Reports

Performance Reports

Medical Reports

Coach Reports

Artist Reports

Audit Reports

Export CSV

Export PDF

Reporting should remain efficient.

Notification Module

Support

System Notifications

Attendance Notifications

Performance Updates

Medical Alerts

Activity Updates

Administrative Messages

Validation

Validate

Body

Query

Route Parameters

Headers

Files

Use Zod.

Never process invalid requests.

Error Handling

Implement

Validation Errors

Authentication Errors

Authorization Errors

Business Errors

Database Errors

Unexpected Errors

Return consistent responses.

Never expose internal details.

Audit Logging

Log

Authentication

Authorization Failures

Attendance Changes

Medical Updates

Role Changes

Administrative Actions

Security Events

Audit records must be immutable.

Transactions

Use Prisma transactions for

Attendance

Medical Updates

Role Changes

Activity Registration

Profile Updates

Performance Records

Audit Logging

Prevent partial updates.

Performance

Optimize

Queries

Filtering

Sorting

Pagination

Indexes

Caching

Response Time

Avoid N+1 queries.

Security

Protect against

SQL Injection

Cross Site Scripting

CSRF

Broken Authentication

Broken Authorization

Session Hijacking

IDOR

Mass Assignment

Sensitive Data Exposure

Follow OWASP Top 10.

Documentation

Document every API.

Purpose

Authentication

Authorization

Validation

Request

Response

Errors

Examples

Security Notes

Code Quality

Every module should

Compile successfully.

Follow TypeScript strict mode.

Pass ESLint.

Pass formatting.

Avoid duplication.

Remain readable.

Support future expansion.

Deliverables

Complete Backend

Authentication

Authorization

REST APIs

Server Actions

Business Services

Repositories

Validation

Audit Logging

API Documentation

Error Handling

Transactions

Performance Review

Security Review

Acceptance Criteria

Every API works.

Authentication is complete.

Authorization is enforced.

Validation exists.

Business rules are centralized.

Transactions work correctly.

Audit logs are generated.

Performance is acceptable.

Documentation is complete.

Tests are ready.

Update Project Memory

Mark

Backend Development

Completed

Record

Modules Implemented

API Count

Business Rules

Known Issues

Technical Debt

Performance Notes

Next Task

Frontend Development

Backend Review Checklist

Verify

All endpoints compile.

Authentication works.

Authorization works.

Validation is complete.

Transactions succeed.

Audit logs exist.

Performance targets are met.

Security controls are implemented.

Documentation is complete.

No critical defects remain.

Definition of Done

The backend is production ready.

Every module is implemented.

Every endpoint is secure.

Every business rule is enforced.

The architecture remains clean.

The code is maintainable.

The APIs are documented.

The backend is approved for frontend integration and security testing.